

Documentação do Projeto

Sistema de Otimização de Rotas com Algoritmo Genético e LLM

Aluno	Fernando Monin
Instituição	FIAP (Faculdade de Informática e Administração Paulista)
Cidade e ano	São Paulo, março de 2026
GIT	https://github.com/fmonin/tech_challenge_ia_genetico_tsp.git
Vídeo	https://youtu.be/aT6vNuB36Ms

1. Introdução

Este trabalho apresenta uma solução acadêmica para otimização de rotas inspirada no Travelling Salesman Problem (TSP, ou Problema do Caixeiro Viajante) e adaptada para um cenário próximo do Vehicle Routing Problem (VRP, ou Problema de Roteamento de Veículos). A proposta usa cidades da região de São Paulo e uma frota com três veículos de perfis diferentes, buscando uma distribuição de entregas mais equilibrada.

A ideia central foi usar um Algoritmo Genético (AG), que é uma técnica de Inteligência Artificial inspirada em evolução natural. No projeto, o AG procura combinações de rotas que reduzam a distância total, o custo operacional e as penalidades por restrições, sem deixar de considerar a prioridade de cidades mais críticas.

Além da parte de otimização, o projeto também possui uma camada com Large Language Model (LLM, ou Modelo de Linguagem de Grande Porte). Essa camada ajuda a transformar o resultado técnico em relatórios, respostas em linguagem natural e recomendações operacionais mais fáceis de interpretar.

2. Visão geral do projeto

De forma simples, o arquivo principal executa o problema de rotas, avalia várias soluções ao longo das gerações e mostra a evolução em uma tela visual. Depois disso, o histórico gerado pode ser consultado por uma Interface do Usuário (UI) construída com Streamlit. Essa parte web permite fazer perguntas sobre a última execução sem depender da janela gráfica.

Tema do projeto	Otimização de rotas com Algoritmo Genético (AG) e apoio de LLM (Modelo de Linguagem de Grande Porte).
Problema tratado	Distribuir cidades entre três veículos, respeitando capacidade, distância máxima, tempo de trabalho e prioridade das entregas.
Tecnologias usadas	Python, Pygame, Streamlit, Matplotlib, JSON (JavaScript Object Notation) e OpenAI API (Application Programming Interface).
Entrada principal	Base de cidades, regras da frota, parâmetros do algoritmo e histórico salvo da última execução.
Saídas principais	Mapa das rotas, resumo operacional, gráfico de evolução, histórico em arquivo e respostas em linguagem natural.

3. Documentação detalhada

A arquitetura foi separada em camadas para deixar cada parte com uma responsabilidade mais clara. Essa escolha ajuda na manutenção do código e melhora a apresentação acadêmica, porque fica mais fácil explicar onde acontece a otimização, onde acontece a visualização e onde acontece a consulta com linguagem natural.

3.1 Arquitetura geral da solução

No fluxo principal, o arquivo `tsp.py` é o núcleo do problema. Ele carrega as cidades, define os veículos, chama os operadores genéticos, calcula a fitness e atualiza a tela gráfica. Já o módulo `draw_functions.py` fica responsável apenas pela parte visual. Em paralelo, a consulta web usa os módulos `app_streamlit.py`, `llm_ui_service.py` e `route_context_loader.py` para reaproveitar os resultados sem abrir a interface gráfica.

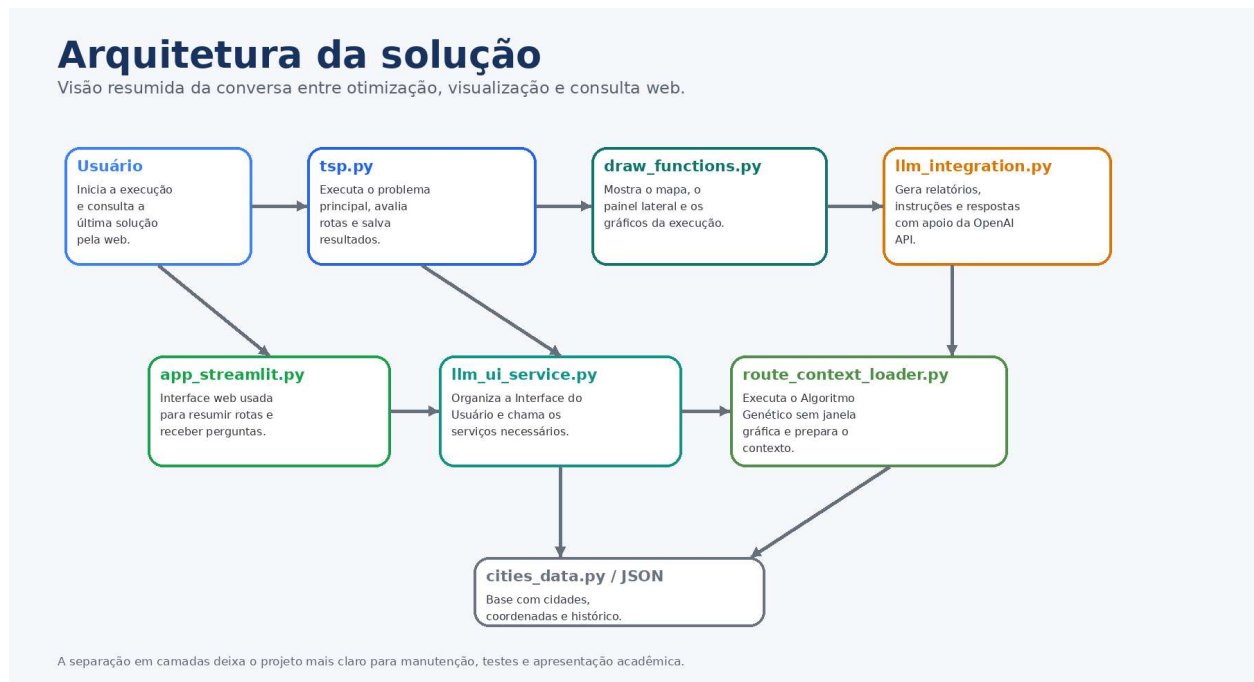


Figura 1 - Arquitetura resumida da solução.

Na imagem acima, dá para perceber que a solução foi dividida entre execução principal, visualização, consulta web e geração de relatórios. Essa divisão reduz o acoplamento, ou seja, evita que uma parte dependa demais da outra.

3.2 Principais módulos

- `tsp.py`: arquivo principal. Monta as cidades, configura a frota, avalia rotas e controla a evolução das gerações.
- `genetic_algorithm.py`: implementa seleção por torneio, cruzamento, mutação e melhoria local com 2-opt.
- `draw_functions.py`: desenha o mapa, o painel lateral e os gráficos usados para acompanhar a execução.
- `llm_integration.py`: transforma os resultados em relatórios, instruções e respostas em linguagem natural.
- `llm_ui_service.py` e `route_context_loader.py`: fazem a ponte entre a consulta web e a execução do algoritmo sem depender da janela gráfica.
- `cities_data.py` e JSON (JavaScript Object Notation): guardam base de cidades, coordenadas e histórico das execuções.

3.3 Diagrama de componentes

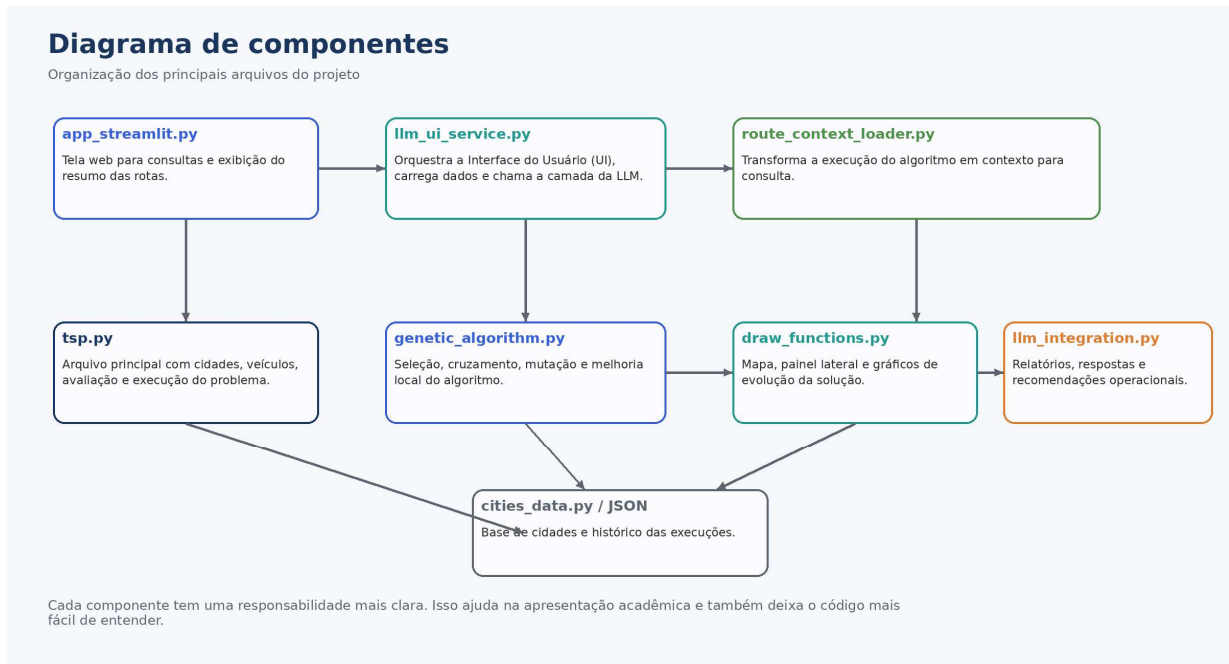


Figura 2 - Diagrama de componentes do projeto.

O diagrama de componentes complementa a arquitetura geral porque mostra a organização dos arquivos do projeto. Nesse desenho, fica mais fácil identificar o papel de cada módulo e o caminho dos dados entre eles.

4. Metodologia de Algoritmos Genéticos aplicada ao problema

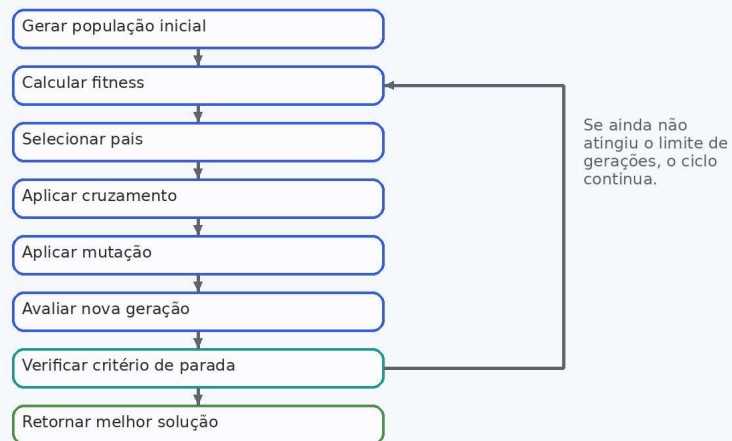
O Algoritmo Genético (AG) foi escolhido porque o problema de rotas possui muitas combinações possíveis. Em vez de testar tudo de forma exata, o AG busca boas soluções aos poucos, comparando indivíduos, selecionando os melhores e criando novas combinações ao longo das gerações.

Neste projeto, cada indivíduo representa uma permutação global das cidades. Depois dessa ordem ser montada, um processo de decodificação distribui as cidades entre os veículos de acordo com a capacidade, a distância máxima, o tempo de trabalho e a prioridade de atendimento. Esse detalhe é importante porque aproxima a solução de uma situação logística mais real.

- População inicial: várias sequências de cidades são geradas de forma aleatória.
- Fitness: cada solução recebe uma nota que considera distância, custo operacional, prioridade e penalidades por restrições.
- Seleção: o projeto usa seleção por torneio, escolhendo os indivíduos mais promissores para gerar filhos.
- Cruzamento: foi usado Order Crossover (OX, ou Cruzamento por Ordem), adequado para problemas de permutação.
- Mutação: o algoritmo aplica troca de posições ou inversão de trechos para aumentar a diversidade.
- Melhoria local: parte das soluções passa por 2-opt para reduzir cruzamentos e melhorar o caminho.
- Critério de parada: a execução termina ao atingir o número configurado de gerações.

Fluxo do Algoritmo Genético

Etapas principais da busca por uma boa solução de rotas



Nesse projeto, a melhor solução considera distância, custo operacional, prioridade das cidades e penalidades por restrições.

Figura 3 - Fluxo simplificado do Algoritmo Genético.

Na prática, a melhor solução não é só a mais curta. Ela também precisa respeitar limites da operação. Por isso, a função fitness é multicritério e tenta encontrar um equilíbrio entre custo, prioridade e viabilidade da rota.

5. Desenvolvimento da solução

A implementação foi organizada para separar o que é execução do problema e o que é consulta posterior. O arquivo `tsp.py` concentra o ciclo principal do problema, enquanto `draw_functions.py` cuida da renderização. Já a camada com Streamlit reaproveita o histórico salvo e executa um fluxo mais leve para consulta.

Os parâmetros principais observados no código foram população igual a 90, 15 gerações, probabilidade de mutação de 0,22, elitismo igual a 2 e torneio com 6 indivíduos. Esses valores mostram uma preocupação em equilibrar qualidade da solução e tempo de execução.

Entradas	Cidades, coordenadas, demandas, regras dos veículos e parâmetros do Algoritmo Genético.
Processamento	Geração da população, avaliação de fitness, seleção, cruzamento, mutação e melhoria local.
Saídas	Rotas por veículo, resumo operacional, gráfico de evolução e histórico salvo em arquivo.
Consulta web	Perguntas do usuário, leitura do histórico e respostas produzidas pela camada de LLM.

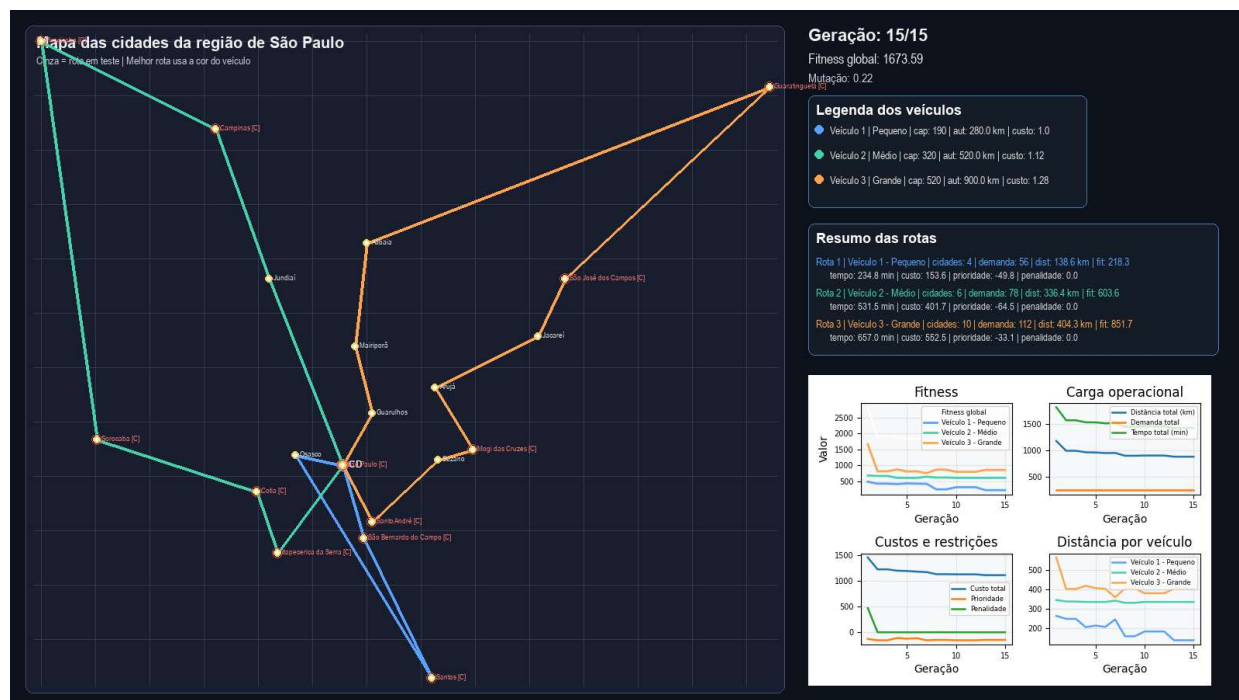


Figura 4 - Exemplo de execução principal com mapa, legenda dos veículos, resumo das rotas e gráficos laterais.

O print acima mostra a solução funcionando de forma integrada. No mesmo painel aparecem o mapa, as cores das rotas, o resumo por veículo e a evolução das métricas operacionais.

6. Resultados obtidos ou esperados

Pelos resultados mostrados nas imagens geradas pelo projeto, a solução consegue distribuir as cidades entre os três veículos e apresentar uma leitura visual clara da operação. Isso é útil tanto para análise técnica quanto para apresentação acadêmica.

- redução progressiva da fitness global ao longo das gerações;
- equilíbrio entre distância total, custo e atendimento das cidades críticas;
- separação visual das rotas por veículo, o que facilita a interpretação;
- produção de histórico para consulta posterior pela interface web e pela camada de LLM.

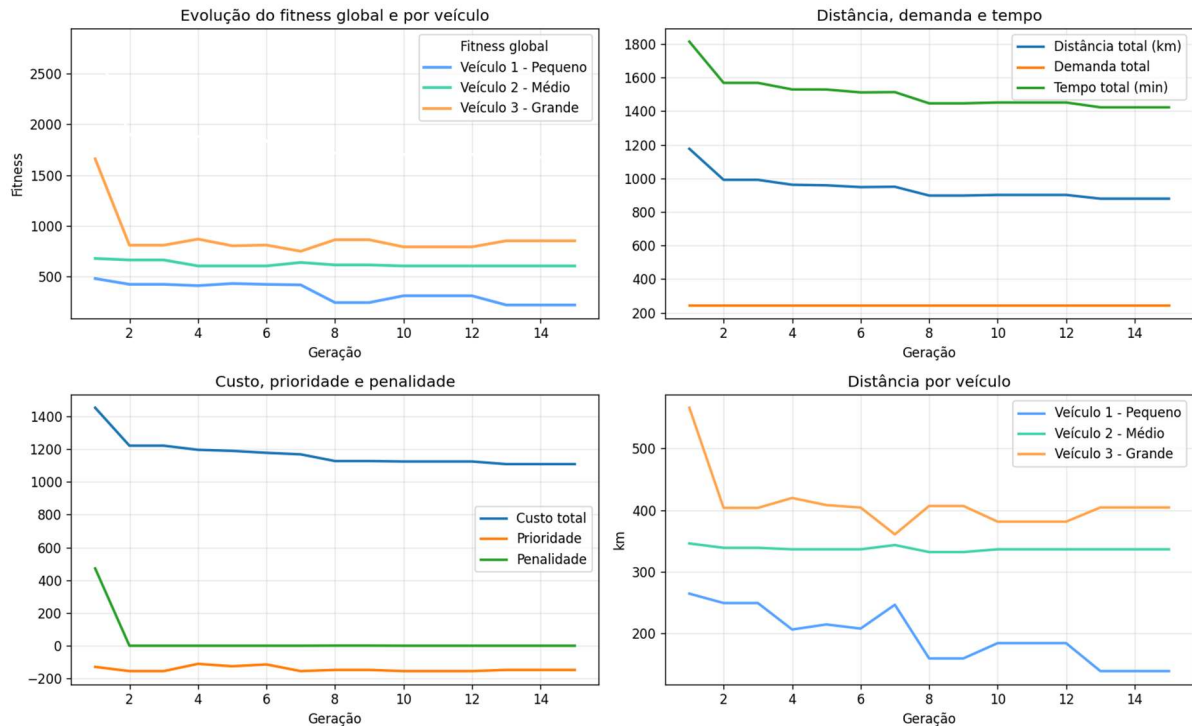


Figura 5 - Gráficos de evolução da fitness, distância, demanda, tempo, custo e distância por veículo.

Nos gráficos, é possível notar que o comportamento fica mais estável depois das primeiras gerações. Isso sugere que o algoritmo encontra soluções melhores no começo e depois passa a trabalhar mais no refinamento. Mesmo sendo uma abordagem heurística, o resultado final fica consistente para o objetivo do projeto.

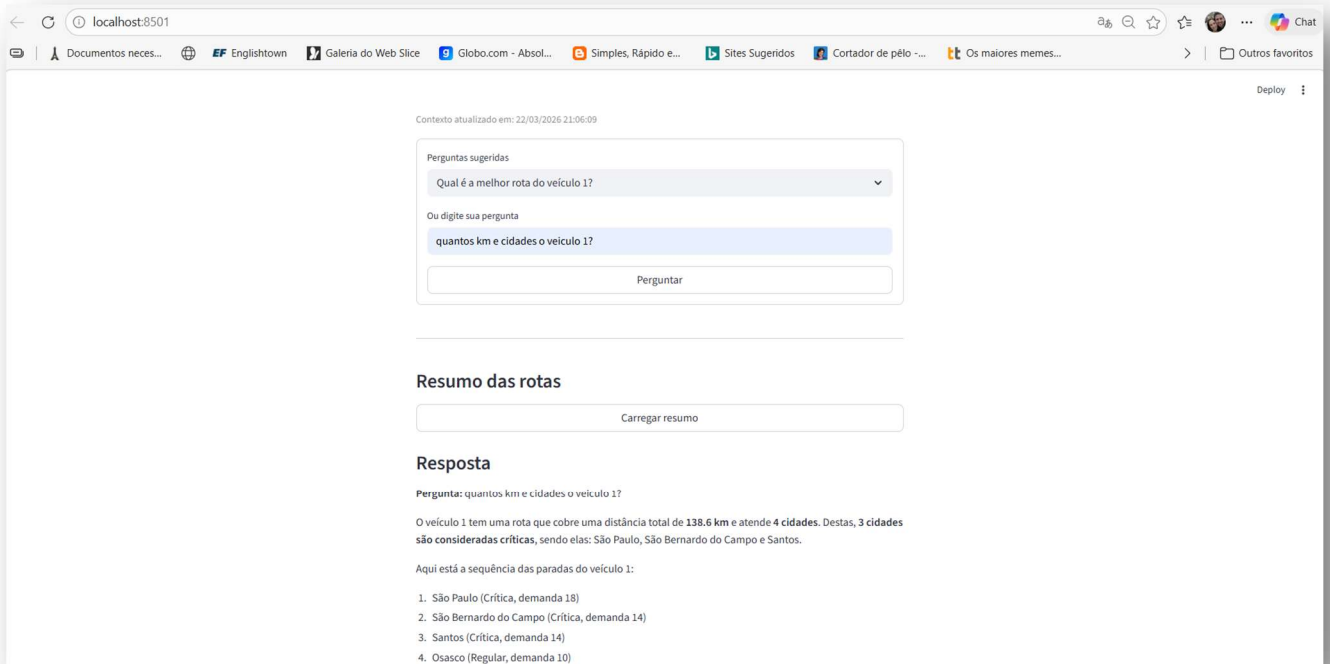
6.1 Limitações observadas

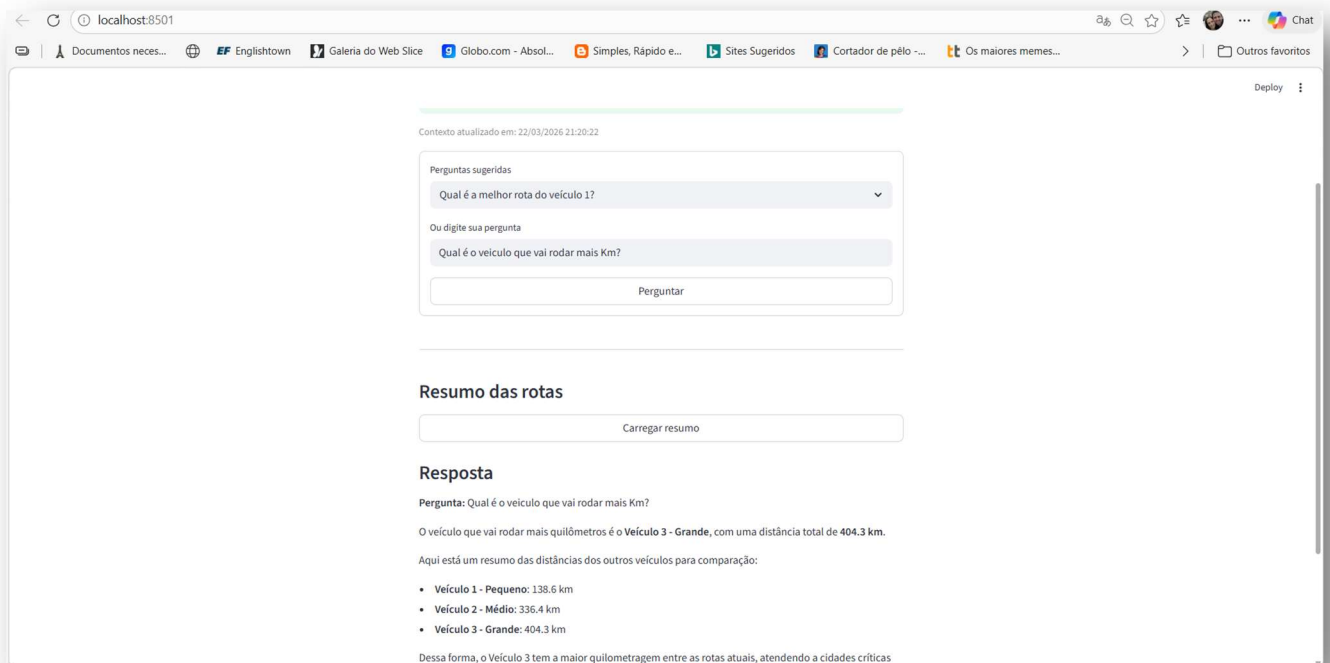
- o modelo ainda é uma simplificação do VRP (Problema de Roteamento de Veículos) real;
- o número de cidades e de veículos é controlado para manter a execução em tempo razoável;
- a qualidade final depende dos parâmetros escolhidos para população, gerações e mutação;
- a camada de LLM depende de chave de acesso e de um histórico salvo corretamente.

7. Conclusão

Este projeto mostrou, de forma prática, como um Algoritmo Genético (AG) pode ser usado para resolver um problema de otimização de rotas com múltiplos critérios. A solução não trabalha apenas com menor distância, mas também considera custo operacional, prioridade das cidades e penalidades por restrições.

Outro ponto relevante foi a combinação entre busca heurística e interpretação em linguagem natural. A presença do Large Language Model (LLM, ou Modelo de Linguagem de Grande Porte) deixa o resultado mais acessível para consulta, porque transforma números e rotas em relatórios e respostas mais fáceis de entender.





Como trabalho de pós-graduação, o projeto ficou interessante porque une teoria e prática. De um lado, existe o estudo da computação evolucionária. Do outro, existe uma implementação funcional com interface visual, histórico persistido e consulta web. Para evoluções futuras, seria possível ampliar a base de cidades, testar outros operadores genéticos e comparar o desempenho com outras técnicas de otimização.